УНИВЕРСИТЕТ ИТМО
Факультет Систем Управления и Робототехники

Лабораторная работа №3 «Частотные методы»

Частотные методы

Выполнили:
Бурдасова Виктория: 465306
Жаворонков Пётр: 465877
Беляев Александр: 465203

Санкт-Петербург
2026

Содержание

1 Фильтрация звука **1**
1.1 Постановка задачи 2
1.2 Ход работы 2
1.3 Вывод 3

2 Приложения **3**
2.1 Код к заданию 2 4

1. Фильтрация звука

1.1 Постановка задачи

Загрузить с диска файл `MUHA.wav` и при помощи фильтрации Фурье-образа очистить сигнал от шумов, оставив только голос.

Для преобразования аудиофайла в массив со значениями сигнала использовалась Python-библиотека librosa. Для Фурье-преобразований были использованы методы `signal.fft: fft, fftshift, ifft, ifftshift.` Для воспроизведения полученного сигнала как аудиозаписи был использован метод `sounddevice.play()` (библиотека sounddevice).

1.2 Ход работы

Опустим вывод аудиофайла как графика функции сигнала и сразу перейдём к модулю образа. Далее под образом будет подразумеваться модуль сдвинутого образа, полученный путём преобразования `img = fftshift(fft(signal))`. Образ исходника выглядит следующим образом:

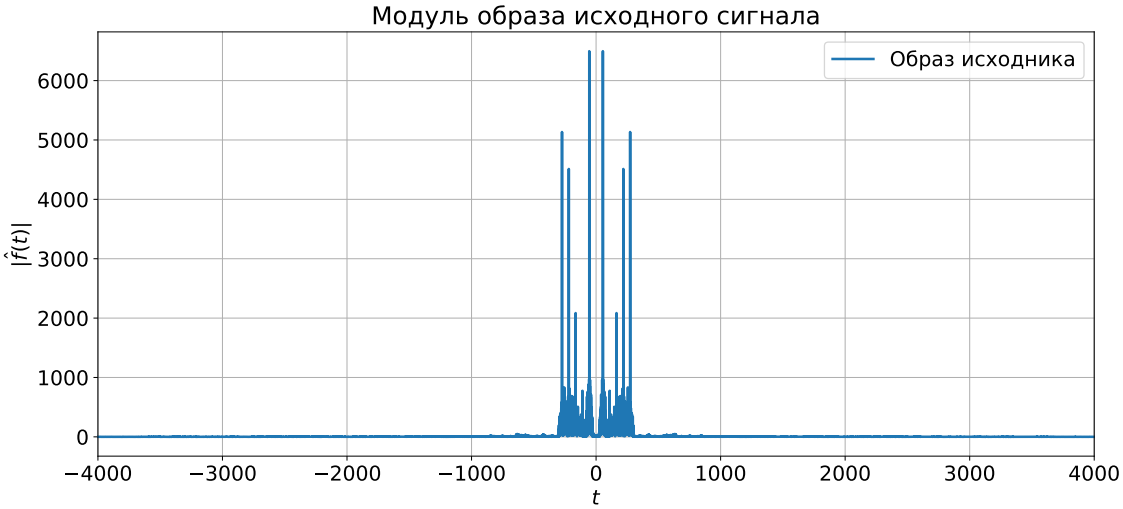

Рис. 1: Модуль образа исходного сигнала

Перед нами явно была поставлена задача: отделить речь в записи от шумов. Для выделения речи мы решили обратиться к стандартным в телефонии значениям и положить, что речевая полоса частот лежит в диапазоне от 300 Гц до 3400 Гц. В связи с тем, что целевой диапазон нам известен, выбор пал на полосовой фильтр: мы отсекаем всё, кроме нужного нам диапазона.

Реализация фильтра в `Python` выглядит следующим образом:

```
LOW_FREQ = 300
HIGH_FREQ = 3400
mask = numpy.zeros_like(img_muha_f)
ind = numpy.where((np.abs(freq) < HIGH_FREQ) & (np.abs(freq) > LOW_FREQ))
mask[ind]=1
filtered_muha_img = img_muha_f * mask
```

Если посмотреть на модуль отфильтрованного образа (рис. 2) сразу видны колоссальные изменения: в связи с тем, что многие низкочастотные шумы имели очень высокие амплитуды,

после фильтрации осталась только низкоамплитудная информативная часть (максимум уменьшился на 2 порядка).

Рис. 2: Модуль образа исходного сигнала

То же самое видно при сравнении графиков сигнала (рис. 3) до и после фильтрации.

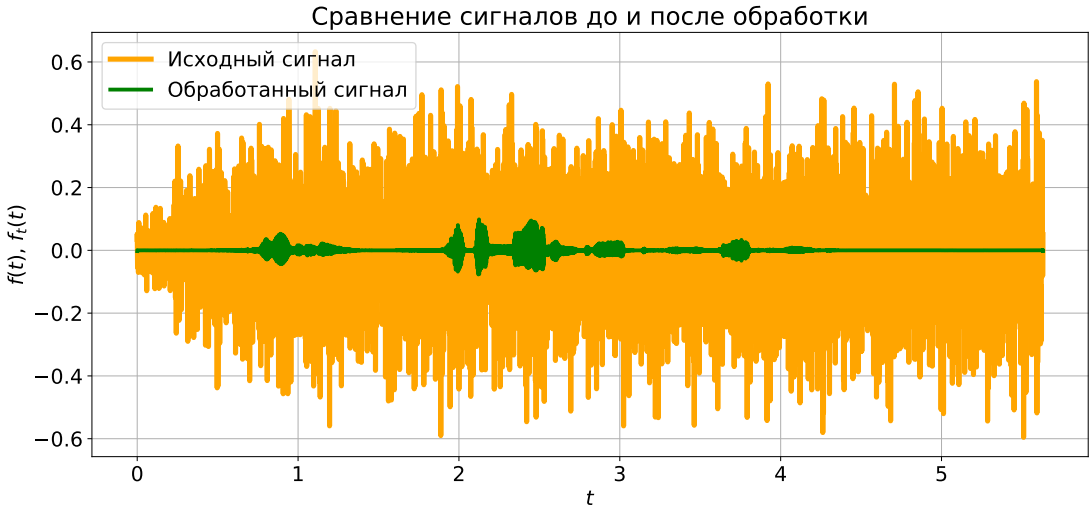

Рис. 3: Модуль образа исходного сигнала

1.3 Вывод

При прослушивании получившейся аудиозаписи отчётливо слышно фразу «Муха – это маленькая птичка!», причем отсутствуют в целом какие-то аудиоартефакты, шумы. Только информация о мухе.

2. Приложения

2.1 Код к заданию 2

```
from scipy.fft import fft, fftshift, ifft, ifftshift
import matplotlib.pyplot as plt
import numpy as np

from librosa import load
import sounddevice as sd

plt.rcParams.update({'font.size': 16})

def save_separate_figures(extension="pdf", path=""):
    """ Saves each active matplotlib figure to a separate file. """
    # Получаем номера всех созданных фигур
    fig_nums = plt.get_fignums()

    for i in fig_nums:
        plt.figure(i)
        filename = f"figure_{i}.{extension}"
        plt.savefig(path+filename)
        print(f"Сохранен файл: {filename}")

def setup_plot(figsize=(14, 6)):
    plt.figure(figsize=figsize)
    plt.grid()

muha_f, muha_sr = load("lab3/MUHA.wav", mono=True)
dt = 1/muha_sr
T = len(muha_f)*dt
t = np.linspace(0, T, len(muha_f))

freq = fftshift(np.fft.fftfreq(len(muha_f), dt))

setup_plot()
plt.title("Исходный сигнал MUHA.wav")
plt.plot(t, np.real(muha_f), linewidth=2, label="Исходный сигнал")
plt.legend()
plt.xlabel(r"$t$")
plt.ylabel(r"$f(t)$")
img_muha_f = fftshift(fft(muha_f))

setup_plot()
plt.title("Модуль образа исходного сигнала")
plt.plot(freq, np.abs(img_muha_f), linewidth=2, label="Образ исходника")
plt.xlim([-4000, 4000])
plt.legend()
plt.xlabel(r"$t$")
plt.ylabel(r"|$\hat f(t)$|")
HIGH_FREQ = 3400
LOW_FREQ = 300
```

```
mask = np.zeros_like(img_muha_f)
ind = np.where((np.abs(freq) < HIGH_FREQ) & (np.abs(freq) > LOW_FREQ))
mask[ind]=1
filtered_muha_img = img_muha_f * mask
mask[ind]=max(np.abs(filtered_muha_img))

setup_plot()
plt.title("Модуль образа после применения фильтра")
plt.plot(freq, np.abs(filtered_muha_img), linewidth=2, label="Фильтрованный образ")
plt.plot(freq, mask, linewidth=1.5, color="orange", label="Полосовой фильтр")
plt.xlim([-4000, 4000])
plt.xlabel(r"$t$")
plt.ylabel(r"|$\hat f(t)$|")
plt.legend()

filtered_muha = ifft(ifftshift(filtered_muha_img))

setup_plot()
plt.title("Отфильтрованный сигнал MUHA.wav")
plt.plot(t, np.real(filtered_muha), linewidth=2, label="Отфильтрованный сигнал")
plt.legend()
plt.xlabel(r"$t$")
plt.ylabel(r"$f(t)$")

setup_plot()
plt.title("Сравнение сигналов до и после обработки")
plt.xlabel(r"$t$")
plt.ylabel(r"$f(t)$, $f_t(t)$")

plt.plot(t, np.real(muha_f), linewidth=4, color="orange", label="Исходный сигнал")
plt.plot(t, np.real(filtered_muha), linewidth=3, color="green", label="Обработанный
    ↪ сигнал")
plt.legend()
plt.fill_between(t, np.real(filtered_muha), np.real(muha_f), alpha=0.4,
    ↪ color="orange")

sd.play(np.real(muha_f), muha_sr)
sd.wait()
sd.play(np.real(filtered_muha), muha_sr)
sd.wait()

save_separate_figures(path="lab3/pete-graphs/")
# plt.show()
```